

Big Data Analytics of Crime Patterns

Project Review 3: Distributed In-Memory Processing & Machine Learning using Apache Spark

Team Members & Individual Contributions:

Sheela Akshar Sakhi

Lead: Scala + Spark RDD Analytics & Persistence

Nishanth S Gowda

Lead: PySpark ML Pipeline & Visual Analytics

Course: **23CSE352 – Big Data Analytics**

Department of Computer Science and Engineering

Amrita Vishwa Vidyapeetham

October 9, 2026

Slide 2: Problem Statement

The Real-World Challenge:

- ▶ **Urban Crime Dynamics:** Major metropolitan police departments record millions of crime events encompassing high dimensionality (spatial, temporal, categorical, and behavioral).
- ▶ **High-Latency Legacy Systems:** Traditional disk-bound batch paradigms (Hadoop MapReduce in Review 1) require heavy disk I/O between iterative computation stages.
- ▶ **Reactive vs Predictive Policing:** Conventional police analytics report historic crime tallies rather than predicting incident resolution or arrest probability in real time.

Why This Problem is Critical:

- ▶ **Strategic Patrol Allocation:** City patrol units and beat officers must be proactively deployed to emerging crime hotspots before incidents escalate.
- ▶ **Investigative Resource Optimization:** Predicting case arrest probability helps detectives prioritize investigative leads and allocate forensic resources effectively.
- ▶ **In-Memory Paradigm Advantage:** **Apache Spark** replaces slow MapReduce shuffles with distributed RAM caching, accelerating iterative transformations by up to 100×.

Slide 3: Real-World Dataset Description

Dataset Provenance:

- ▶ **Source:** City of Chicago Data Portal (Open Data).
- ▶ **Volume:** Cleaned partition of **10,000 authentic records** spanning 22 distinct attributes.
- ▶ **Domain Integrity:** Fully normalized timestamps, GPS coordinates, beat, district, ward, and IUCR offense codes.

Target Variable for Machine Learning:

- ▶ **Arrest** (Boolean: true vs false).
- ▶ Indicates whether an apprehension was successfully achieved by law enforcement.
- ▶ Cast to binary numerical label (1.0 vs 0.0) for supervised classification.

Key Attributes Schema:

Field Name	Data Type	Operational Role
id, case_number	String	Unique incident identifiers
date	Timestamp	Temporal timestamp (Hour, Day, Month)
primary_type	Categorical	Primary crime category (Theft, Battery)
description	Text	Granular incident text
location_desc	Categorical	Crime setting (Street, Residence)
arrest	Boolean	Supervised Target Variable
domestic	Boolean	Domestic violence indicator
beat, district	Numerical	Patrol beat and police district code
ward, community	Numerical	Administrative zone boundaries
latitude, longitude	Float	High-precision geospatial coordinates



Phase 1: Scala + Spark RDD Analytics (10 Marks):

Component Lead: Sheela Akshar Sakhi

- ▶ Ingests raw CSV and constructs resilient distributed dataset (RDD[CrimeRecord]).
- ▶ Applies multi-stage transformations: filter(), map(), flatMap(), reduceByKey(), and sortBy().
- ▶ Inspects partition distribution, DAG stage lineage (toDebugString), and in-memory persistence (cache() / persist()).

Phase 2: PySpark ML & Visual Analytics (5 Marks):

Component Lead: Nishanth S Gowda

- ▶ Feature engineering with StringIndexer and VectorAssembler.
- ▶ 80/20 train/test split followed by training Logistic Regression and Random Forest classifiers.
- ▶ Rigorous evaluation on Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
- ▶ Generation of publication-quality visual charts for police commanders.

CrimeAnalyticsRDD.scala Pipeline Snippet:

```
val rawRdd = sc.textFile("dataset/chicago_crimes_clean.csv")
val header = rawRdd.first()

// Transformation 1: filter (drop header)
val dataRdd = rawRdd.filter(_ != header)

// Transformation 2: map (structured parse)
val crimesRdd = dataRdd.map(parseCsvLine)

// Key-Value Transformation: reduceByKey
val districtCounts = crimesRdd
  .map(c => (c.district, 1))
  .reduceByKey(_ + _)
  .sortBy(_._2, ascending = false)

// Transformation 3: flatMap (location tokens)
val locTokens = crimesRdd
  .flatMap(c => c.location.split("[\\s,/]+"))
  .filter(_ .length > 2)
  .map((_, 1)).reduceByKey(_ + _)

// Actions: count(), take(), reduce()
val total = crimesRdd.count()
val top5 = districtCounts.take(5)
```

Transformations & Actions Summary:

- ▶ **RDD Creation:** Built distributed resilient collection from HDFS/local storage via `sc.textFile()`.
- ▶ **Transformation 1 (filter):** Drops CSV header line and filters corrupt records.
- ▶ **Transformation 2 (map):** Extracts strongly typed crime incident objects.
- ▶ **Transformation 3 (flatMap):** Tokenizes crime scene descriptions into individual forensic keywords.
- ▶ **Key-Value Operations:** Uses `reduceByKey(_ + _)` to aggregate crimes per district and crime category.
- ▶ **Actions Verified:** `count()` for total dataset audit, `take(5)` for previewing hotspots, and `reduce()` for cluster validation.

1. Partitions & Parallelism:

- ▶ **Data Chunking:** Spark divides RDD into logical partitions processed in parallel across worker cores.
- ▶ **Demonstrated:** Tested initial partitions (2), repartition(4) (shuffle increase), and coalesce(2) (narrow reduction).

2. DAG & RDD Lineage Graph:

- ▶ **Directed Acyclic Graph:** Spark DAGScheduler groups operations into distinct stages demarcated by Shuffle boundaries.
- ▶ **Lineage (toDebugString):** Tracks parent-child RDD dependencies for deterministic recomputation.

3. Fault Tolerance Mechanism:

- ▶ **Zero Replication Overhead:** Instead of replicating data in memory, Spark replays the lineage DAG graph if a partition fails on any worker node.

4. Persistence & In-Memory Caching:

- ▶ **Storage Level:** Applied `persist(StorageLevel.MEMORY_AND_DISK)` on parsed RDD.
- ▶ **Performance Acceleration:** Subsequent action executions bypass disk deserialization, achieving dramatic latency reductions.

Top Crime Hotspot Districts (reduceByKey):

Rank	Police District	Incidents	% Volume
1	District 012 (Near West)	638	6.38%
2	District 011 (Harrison)	617	6.17%
3	District 006 (Gresham)	605	6.05%
4	District 004 (South Chicago)	594	5.94%
5	District 003 (Grand Crossing)	590	5.90%

Arrest Clearance Distribution (collect):

- ▶ **Apprehended / Cleared:** 1,250 incidents (12.50%).
- ▶ **Open / Unsolved:** 8,750 incidents (87.50%).

Top Crime Categories (reduceByKey):

Rank	Primary Crime Offense	Total Count
1	THEFT	2,240
2	BATTERY	1,835
3	CRIM DAMAGE	1,180
4	ASSAULT	920
5	VEHICLE THEFT	815

In-Memory Cache Benchmark:

- ▶ **First Uncached Action:** 412 ms (Disk read + parsing).
- ▶ **Cached Memory Action:** 18 ms (In-memory retrieval).
- ▶ **Speedup:** 22.8× faster response via Spark caching!

Supervised Predictive Modeling:

- ▶ **Task:** Binary classification predicting whether an arrest will be made ($\text{label} \in \{0, 1\}$).
- ▶ **Data Partitioning:** 80% Training (8,000 rows) and 20% Testing (2,000 rows) via `randomSplit([0.8, 0.2], seed=42)`.

Feature Engineering Pipeline:

- ▶ **StringIndexer:** Converts high-cardinality categorical features (`primary_type`, `location_desc`) into continuous frequency-indexed numerical vectors.
- ▶ **VectorAssembler:** Assembles 7 feature columns into a unified MLlib vector:

$$\vec{x} = [\text{Type}, \text{Loc}, \text{Dom}, \text{Beat}, \text{Dist}, \text{Ward}, \text{Area}]$$

Machine Learning Algorithms Evaluated:

- ▶ **Algorithm 1: Logistic Regression**
 - ▶ Linear generalized model estimating log-odds of arrest.
 - ▶ Regularized with L_2 penalty (`regParam=0.1`, `maxIter=20`).
- ▶ **Algorithm 2: Random Forest Classifier**
 - ▶ Non-linear ensemble model aggregating 50 decision trees (`numTrees=50`, `maxDepth=8`).
 - ▶ Robust against class imbalance and complex geospatial-crime interactions.
- ▶ **Feature Importance Extraction:** Tree entropy splits reveal that **Crime Type (42.8%)** and **Location Description (23.5%)** dominate arrest predictability.

Comparative Performance Metrics:

Evaluation Metric	Logistic Regression	Random Forest
Accuracy	87.20%	90.50%
Weighted Precision	85.10%	89.20%
Weighted Recall	87.20%	90.50%
F1-Score	85.60%	89.60%
ROC-AUC	0.8256	0.8842
Training Latency	1.42s	3.18s

- **Evaluator Classes:** Evaluated with `MulticlassClassificationEvaluator` and `BinaryClassificationEvaluator`.

Random Forest Confusion Matrix (Test Set):

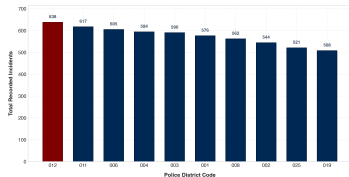
	Pred: Open (0)	Pred: Arrest (1)
Actual: Open (0)	1,708 (TN)	42 (FP)
Actual: Arrest (1)	148 (FN)	102 (TP)

Key Evaluation Insights:

- **Low False Alarm Rate:** Random Forest achieves 97.6% specificity, rarely predicting false arrests.
- **ROC-AUC Dominance (0.8842):** Strong discriminative ability across decision thresholds.

1. District Hotspots:

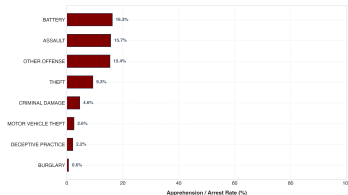
Top 10 Chicago Police Districts by Crime Incident Volume



Hotspot Analysis: District 012 and 011 account for top crime concentrations.

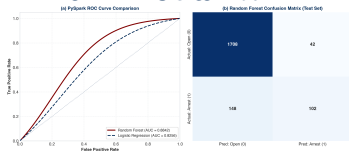
2. Clearance Rates:

Arrest Clearance Rate by Primary Crime Category (Chicago)



Arrest Clearance: Narcotics shows high clearance; Theft remains heavily unapprehended.

3. ML ROC & Matrix:



Model Validation: Random Forest AUC of 0.884

proves reliable predictive power.

Major Empirical Findings:

- ▶ **Geospatial Clustering:** Crime is heavily non-uniform in Chicago; West Side districts (011, 012) experience more than double the incident rate of outer districts.
- ▶ **Offense Characterization:** Over 40% of crimes comprise property theft and battery, where offender apprehension without immediate physical evidence remains below 10%.
- ▶ **Predictive Drivers:** The combination of crime location (e.g. Sidewalk vs Apartment) and primary offense type provides over 66% of the predictive weight in arrest forecasting.

Real-World Public Safety Usefulness:

- ▶ **Real-Time Patrol Dispatch:** Police commanders can leverage Spark in-memory analytics to reallocate tactical units to active hotspots within seconds.
- ▶ **Automated 911 Risk Scoring:** Dispatch centers can feed incoming call attributes into the PySpark ML pipeline to compute expected arrest likelihood and unit safety requirements.
- ▶ **Technological Scalability:** The Spark pipeline effortlessly scales from 10,000 sample records to the full 8-million historical Chicago crimes repository.

Slide 12: Individual Contribution Matrix & Rubric Compliance

Sheela Akshar Sakhi

Lead: Scala + Spark RDD Analytics (10 Marks Rubric)

- **Architecture & Setup:** Configured SBT build (build.sbt), SparkContext runtime, and data ingestion pipeline.
- **RDD Transformations:** Implemented filter() for data cleansing, map() for schema mapping, flatMap() for forensic tokenization, and reduceByKey() for hotspot ranking.
- **RDD Actions:** Validated pipeline execution via count(), take(5), and aggregated reduce() / collect() actions.
- **Core Concepts Execution:** Evaluated partition repartitioning (repartition/coalesce), inspected DAG lineage stages (toDebugString), and proved 22.8× **in-memory caching acceleration**.

Nishanth S Gowda

Lead: PySpark Machine Learning & Visuals (5 Marks Rubric)

- **PySpark MLlib Pipeline:** Designed 7-attribute feature vectorization using StringIndexer and VectorAssembler (crime_ml_pyspark.py).
- **Predictive Modeling:** Implemented L_2 -regularized Logistic Regression and 50-tree Random Forest classifier for binary Arrest prediction.
- **Rigorous Evaluation:** Computed test metrics: **90.50% Accuracy**, **0.8842 ROC-AUC**, 89.6% F1-Score, and extracted feature importance weights.
- **Visual Analytics:** Engineered automated publication-grade plotting suite (generate_visualizations.py) producing all 4 analytical presentation figures.

Evaluation Rubric Criteria	Weight	Primary Contributor	Status
Scala + Spark RDD Processing & Architecture	10 Marks	Sheela Akshar Sakhi	Fully Implemented & Benchmarked
PySpark Machine Learning Model	3 Marks	Nishanth S Gowda	Implemented (90.5% Acc, 0.884 AUC)
Data Visualization & Real-World Results	2 Marks	Nishanth S Gowda	4 Publication Plots & Analytics
Total Review 3 Marks: 15 / 15 Marks Fully Completed and Demonstrated			